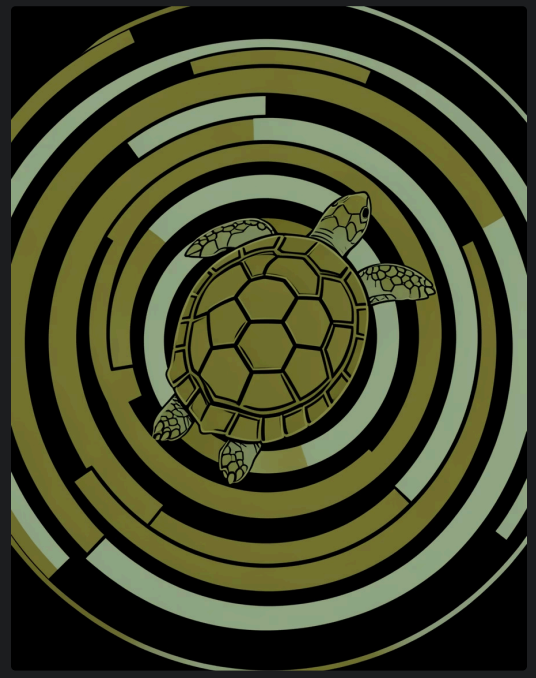
Разбор задания 6 из ЕГЭ по информатике

7 типов задач на Python



Made with GAMMA

Основные команды

forward()

Движение черепахи вперёд на заданное расстояние

left()

Поворот против часовой стрелки на указанный угол

right()

Поворот по часовой стрелке на указанный угол

back()

Движение черепахи назад на заданное расстояние

tracer()

Управление скоростью отрисовки и обновлением экрана

screensize()

Установка размера окна для рисования

goto()

Перемещение черепахи в указанную координату (x, y)

dot()

Рисование точки (круга) в текущей позиции черепахи

up()

Поднятие пера (penup) — движение без рисования

down()

Опускание пера (pendown) — возобновление рисования

done()

Завершение работы программы и сохранение окна

Made with GAMMA

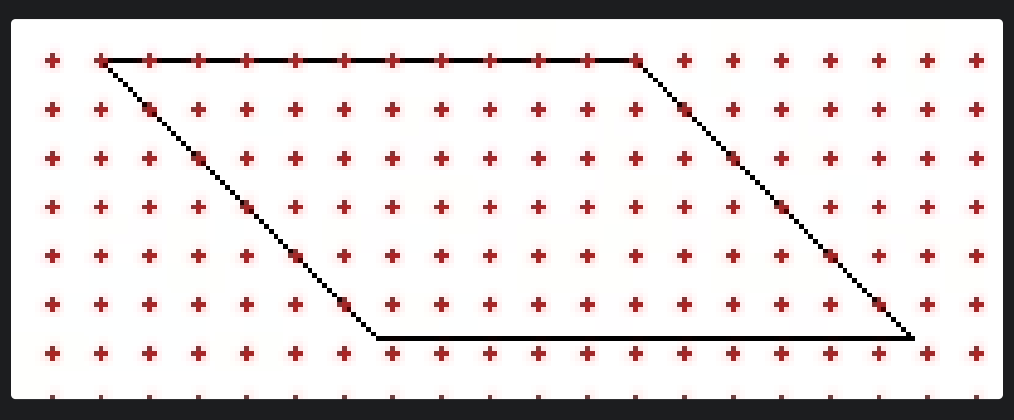
ТИП 1

Простые задачи: Геометрические фигуры

Черепахе был дан для исполнения следующий алгоритм:
Направо 90 Повтори 7 [Вперёд 11 Направо 45 Вперёд 8 Направо 135]
Определите, сколько точек с целочисленными координатами будут находиться внутри области, которая ограничена линией, заданной алгоритмом. Точки на линии учитывать не следует.

Код решения

```
from turtle import *
tracer(0)
c = 10
left(90)
right(90)
for i in range(7):
    forward(11*c)
    right(45)
    forward(8*c)
    right(135)
up()
for x in range(-50,50):
    for y in range(-50,50):
        goto(x*c,y*c)
        dot(3,'red')
done()
```



Зачем нужна переменная c=10?

c = 10 — это коэффициент масштабирования. Он нужен для того, чтобы:

- Увеличить размер фигуры в 10 раз (иначе она будет очень маленькой)
- Сделать фигуру видимой на экране
- Улучшить точность при подсчёте целочисленных точек

Поэтому все значения в командах движения умножаются на c:

- `forward(11*c)` вместо `forward(11)` — движение на 110 пикселей
- `forward(8*c)` вместо `forward(8)` — движение на 80 пикселей

Также при проверке точек используется `goto(x*c, y*c)` — координаты тоже масштабируются, чтобы соответствовать масштабированной фигуре.

Зачем tracer(0)?

`tracer(0)` отключает автоматическое обновление экрана. Это нужно для того, чтобы:

- Ускорить отрисовку (программа работает быстрее)
- Избежать мерцания экрана
- Сразу увидеть готовый результат вместо пошагового рисования

Зачем нужен up()?

`up()` поднимает перо черепахи (penup). После построения фигуры мы поднимаем перо, чтобы:

- Перемещаться по экрану БЕЗ рисования линий
- Проверить каждую точку в области, не оставляя следов
- Только ставить точки (dot) в нужных местах

Вложенные циклы x и y

Двойной цикл нужен для перебора всех целочисленных координат в области:

- Внешний цикл `for x in range(-50,50)` — перебирает все значения x от -50 до 49
- Внутренний цикл `for y in range(-50,50)` — для каждого x перебирает все значения y от -50 до 49
- Итого проверяется 100 × 100 = 10000 точек
- Для каждой точки мы переходим в координату `(x*c, y*c)` и ставим красную точку `dot(3,'red')`

Это позволяет визуализировать всю сетку целочисленных точек и затем вручную подсчитать, сколько из них находится внутри фигуры.

> ⚠️ Важно: Точки на линии считать НЕ следует! Это частая ошибка при решении таких задач. Нужно считать только точки ВНУТРИ фигуры.

Алгоритм решения

1. Построить фигуру по алгоритму
2. Визуализировать все целочисленные точки в области
3. Определить, какие точки находятся внутри (не на границе)
4. Подсчитать количество таких точек

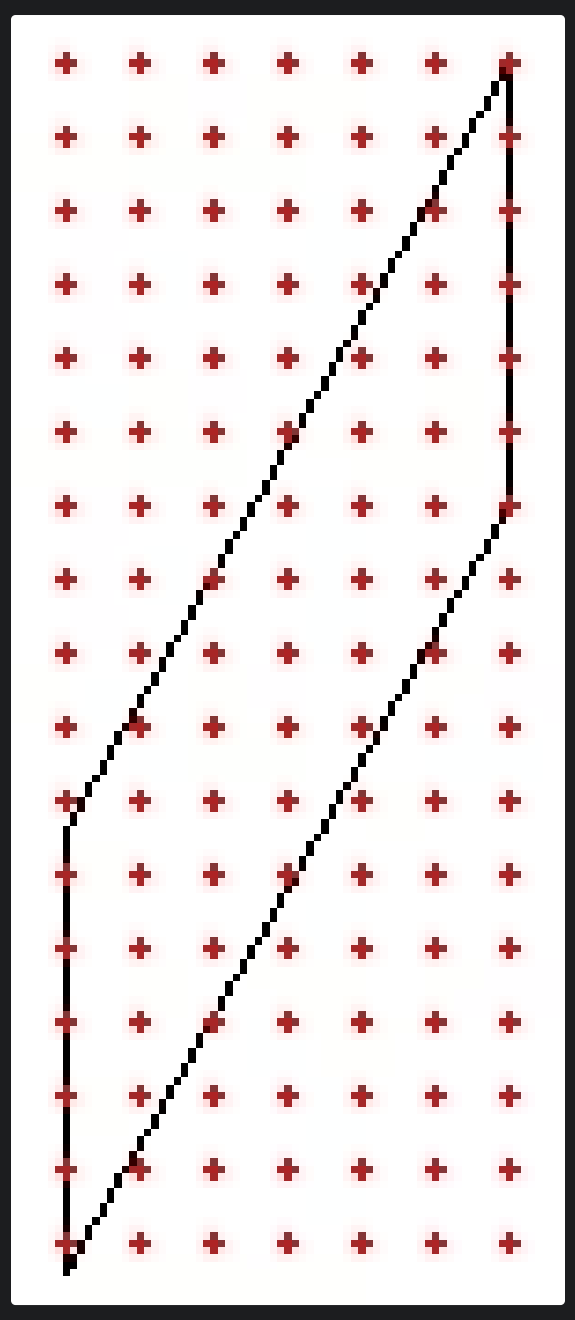
Made with GAMMA

ТИП 2

Спорные точки

Черепахе был дан для исполнения следующий алгоритм: Направо 30 Повтори 3 [Направо 150 Вперёд 6 Направо 30 Вперёд 12] Определите, сколько точек с целочисленными координатами будут находиться внутри области, которая ограничена линией, заданной алгоритмом. Точки на линии учитывать не следует.

```
from turtle import *
tracer(0)
c = 10
left(90)
right(30)
for i in range(3):
    right(150)
    forward(6*c)
    right(30)
    forward(12*c)
up()
for x in range(-50,50):
    for y in range(-50,50):
        goto(x*c,y*c)
        dot(3,'red')
done()
```



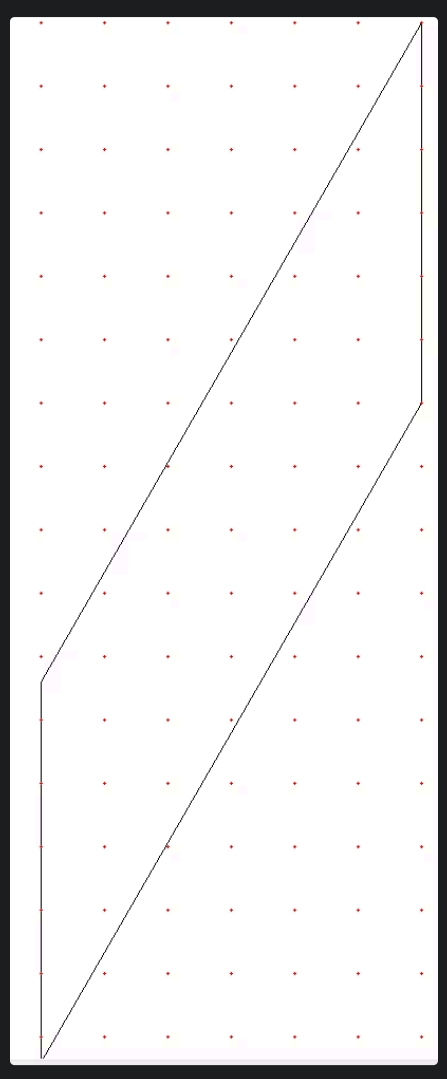
В задачах этого типа требуется определить, какие точки с целочисленными координатами попадают внутрь или на границу фигуры, построенной черепахой.

> **Совет:** Визуализируйте точки красным цветом с помощью метода dot() модуля turtle для наглядности проверки

На данном рисунке есть несколько точек, которые являются спорными, поскольку не ясно, находятся ли они на линии или нет. Для более точного определения увеличим значение с (важно заметить, что значения для с подбирались вручную).

В нашу программу добавим screensize() — значения для которого также подбирались вручную. В результате программа решения выглядит следующим образом:

```
from turtle import *
tracer(0)
screensize(2000,4000)
c = 60
left(90)
right(30)
for i in range(3):
    right(150)
    forward(6*c)
    right(30)
   forward(12*c)
up()
for x in range(-50,50):
    for y in range(-50,50):
        goto(x*c,y*c)
        dot(3,'red')
done()
```



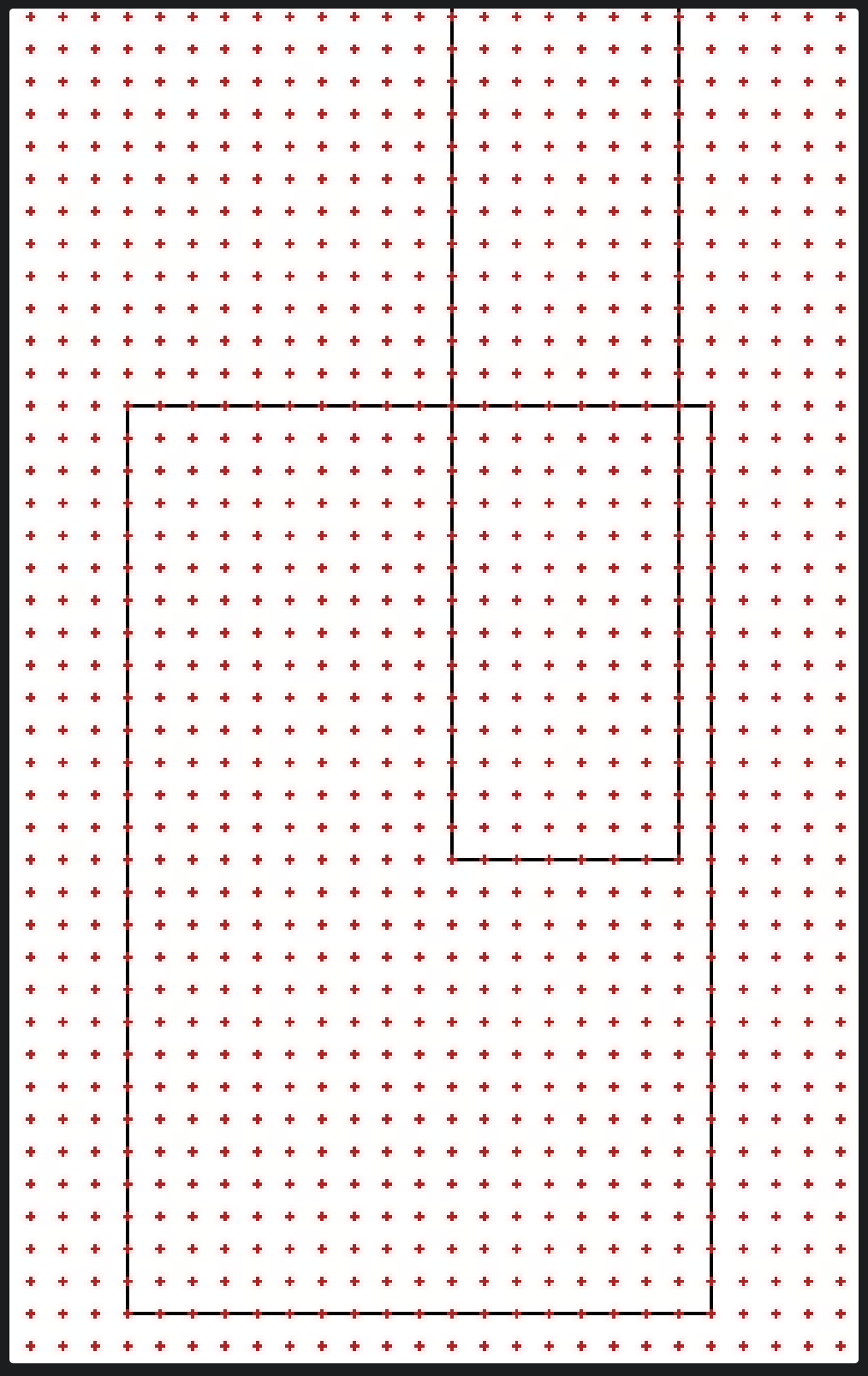
Made with GAMMA

Объединения фигур

Черепахе был дан для исполнения следующий алгоритм: Повтори 2 [Вперёд 28 Направо 90 Вперёд 18 Направо 90] Поднять хвост Вперёд 14 Направо 90 Вперёд 10 Налево 90 Опустить хвост Повтори 2 [Вперёд 30 Направо 90 Вперёд 7 Направо 90] Определите, сколько точек с целочисленными координатами будут находиться внутри объединения фигур, ограниченного заданными алгоритмом линиями, включая точки на линиях.

Решение:

```
from turtle import *
tracer(0)
c = 10
left(90)
for i in range(2):
    forward(28*c)
    right(90)
    forward(18*c)
    right(90)
up()
forward(14*c)
right(90)
forward(10*c)
left(90)
down()
for i in range(2):
    forward(30*c)
    right(90)
    forward(7*c)
    right(90)
up()
for x in range(-50,50):
    for y in range(-50,50):
        goto(x*c,y*c)
        dot(3,'red')
done()
```



Анализ задачи:

Из условия задачи мы знаем размеры двух прямоугольников:

- Первая фигура: 28 × 18
- Вторая фигура: 30 × 7

Важное замечание о подсчёте точек:
При подсчёте целочисленных точек внутри прямоугольника размером a × b количество точек всегда будет (a + 1) × (b + 1). Это происходит потому, что мы считаем точки на обеих границах включительно. Например, для отрезка длины 28 есть 29 целочисленных точек (от 0 до 28 включительно).

Объединение фигур:
Объединение двух фигур — это множество всех точек, которые принадлежат хотя бы одной из этих фигур.

Формула для подсчёта:
Количество точек в объединении = Количество точек в фигуре 1 + Количество точек в фигуре 2 − Количество точек в пересечении

Это необходимо, чтобы не считать точки пересечения дважды.

Вычисление:

Подсчитаем количество точек в каждой фигуре:

- Первый прямоугольник (28 + 1) × (18 + 1) = 29 × 19 = 551 точка
- Второй прямоугольник (30 + 1) × (7 + 1) = 31 × 8 = 248 точек

Определяем размер пересечения:
Из алгоритма видно, что второй прямоугольник смещён на 14 вперёд и 10 вправо от первого. Пересечение имеет размеры 8 × 15.

- Пересечение: 8 × 15 = 120 точек

Применяем формулу объединения:
Количество точек в объединении = 551 + 248 − 120 = 679 точек

Made with GAMMA

Пересечения фигур

Черепахе был дан для исполнения следующий алгоритм:

Повтори 6 [Вперёд 33 Направо 90 Вперёд 20 Направо 90]

Поднять хвост

Вперёд 3 Направо 90 Вперёд 9 Налево 90

Опустить хвост

Повтори 6 [Вперёд 24 Направо 90 Вперёд 25 Направо 90]

Определите, сколько точек с целочисленными координатами находится внутри пересечения фигур, ограниченных заданными алгоритмом линиями. Точки на границах этого пересечения учитывать не следует.

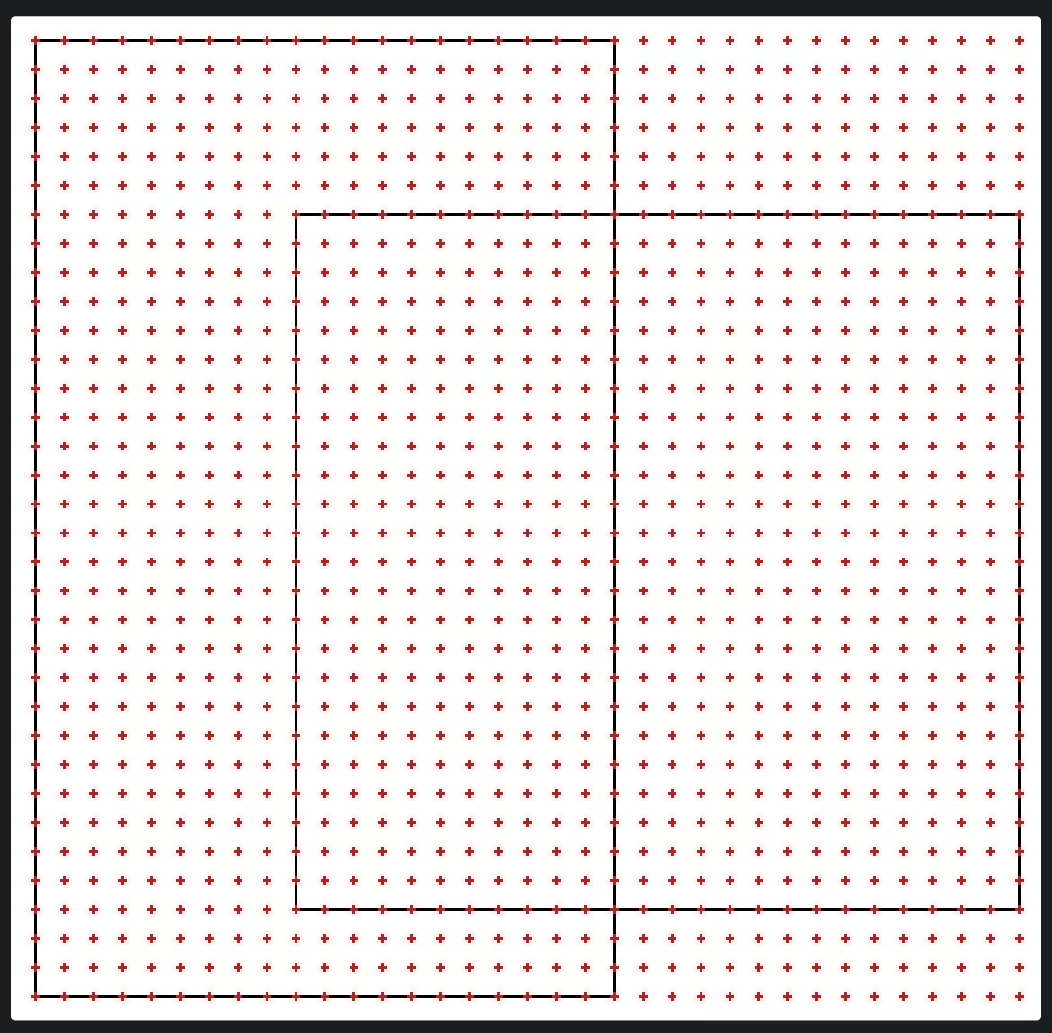
Пересечение фигур:

Пересечение двух фигур — это множество всех точек, которые одновременно принадлежат обеим фигурам.

⚠️ Важное замечание:
В этой задаче требуется подсчитать точки ТОЛЬКО внутри пересечения, исключая точки на границах пересечения. Это отличается от предыдущих задач, где граничные точки учитывались.

Решение:

```
from turtle import *
tracer(0)
c = 10
left(90)
for i in range(6):
    forward(33*c)
    right(90)
    forward(20*c)
    right(90)
up()
forward(3*c)
right(90)
forward(9*c)
left(90)
down()
for i in range(6):
    forward(24*c)
    right(90)
    forward(25*c)
    right(90)
up()
for x in range(-50,50):
    for y in range(-50,50):
        goto(x*c,y*c)
        dot(3,'red')
done()
```



Подсчёт:

Размер пересечения: 10 × 23

Количество точек внутри пересечения (без границ):
10 × 23 = 230

Ответ: 230

Made with GAMMA

Периметр пересечения фигур

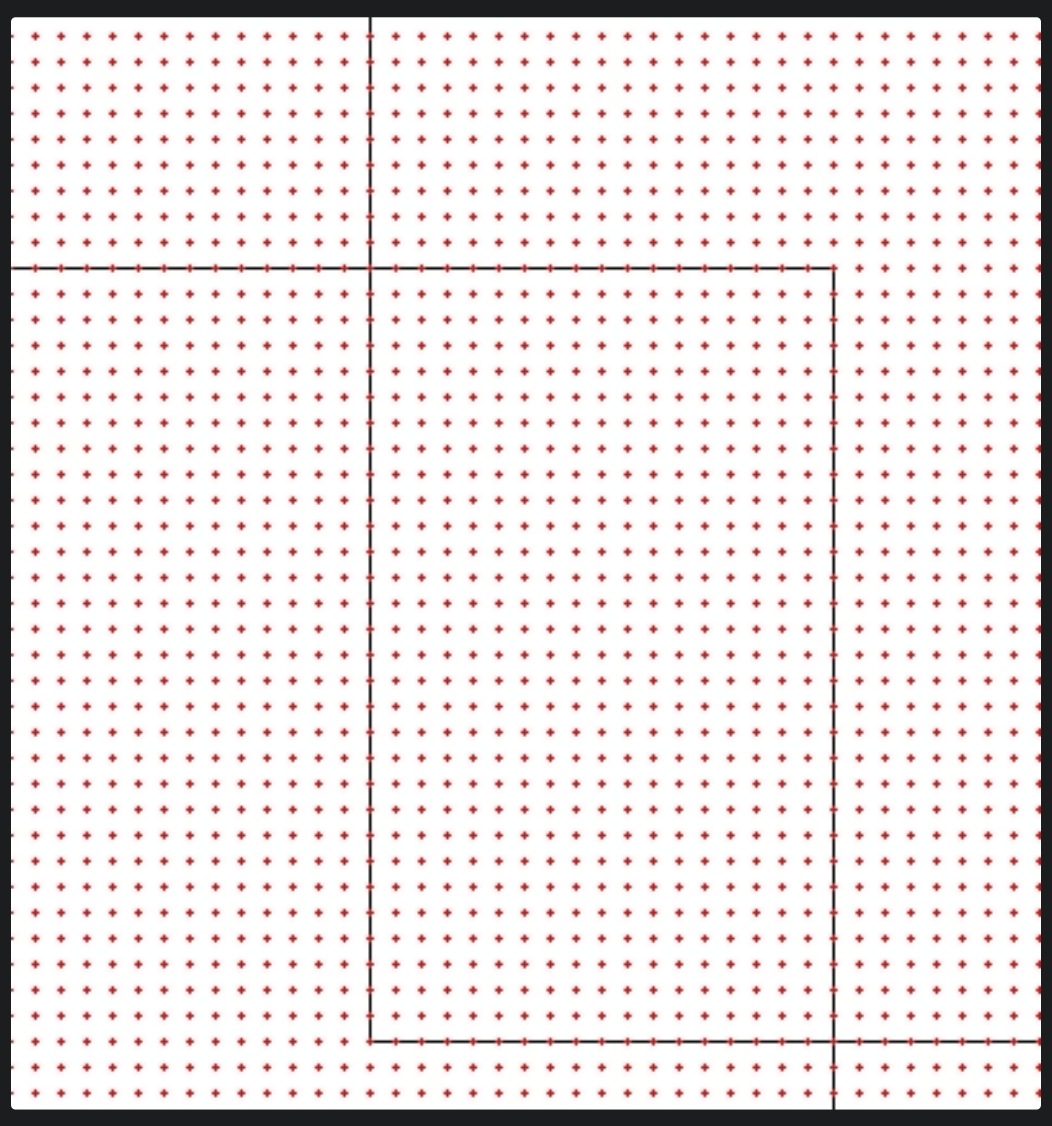
Черепахе был дан для исполнения следующий алгоритм:
Направо 180
Повтори 9 [Вперёд 45 Налево 90 Вперёд 91 Налево 90]
Поднять хвост
Вперёд 15 Налево 90 Вперёд 18 Направо 90
Опустить хвост
Повтори 9 [Вперёд 128 Направо 90 Вперёд 92 Направо 90]
Определите периметр области пересечения фигур, ограниченных заданными алгоритмом линиями.

Важное замечание о периметре и площади:

Когда мы считаем периметр или площадь фигуры, мы уже не считаем отдельные точки, а считаем линии между точками. Периметр — это длина границы фигуры, а площадь — это количество единичных квадратов, которые находятся внутри фигуры. Каждый единичный квадрат образуется между четырьмя соседними целочисленными точками.

Решение:

```
from turtle import *
tracer(0)
screensize(2000,2000)
c = 10
left(90)
right(180)
for i in range(6):
    forward(45*c)
    left(90)
    forward(91*c)
    left(90)
up()
forward(15*c)
left(90)
forward(18*c)
right(90)
down()
for i in range(6):
    forward(128*c)
    right(90)
    forward(92*c)
    right(90)
up()
for x in range(-50,50):
    for y in range(-50,50):
        goto(x*c,y*c)
        dot(3,'red')
done()
```



Примечание: Мы добавляем screensize(2000,2000) для того, чтобы полностью увидеть зону пересечения обеих фигур на экране.

Вычисление:

Размер пересечения: 18 × 30

Периметр пересечения:
18 × 2 + 30 × 2 = 36 + 60 = 96

Ответ: 96

Примечание: На ЕГЭ не встречаются задачи на подсчёт периметра объединения фигур.

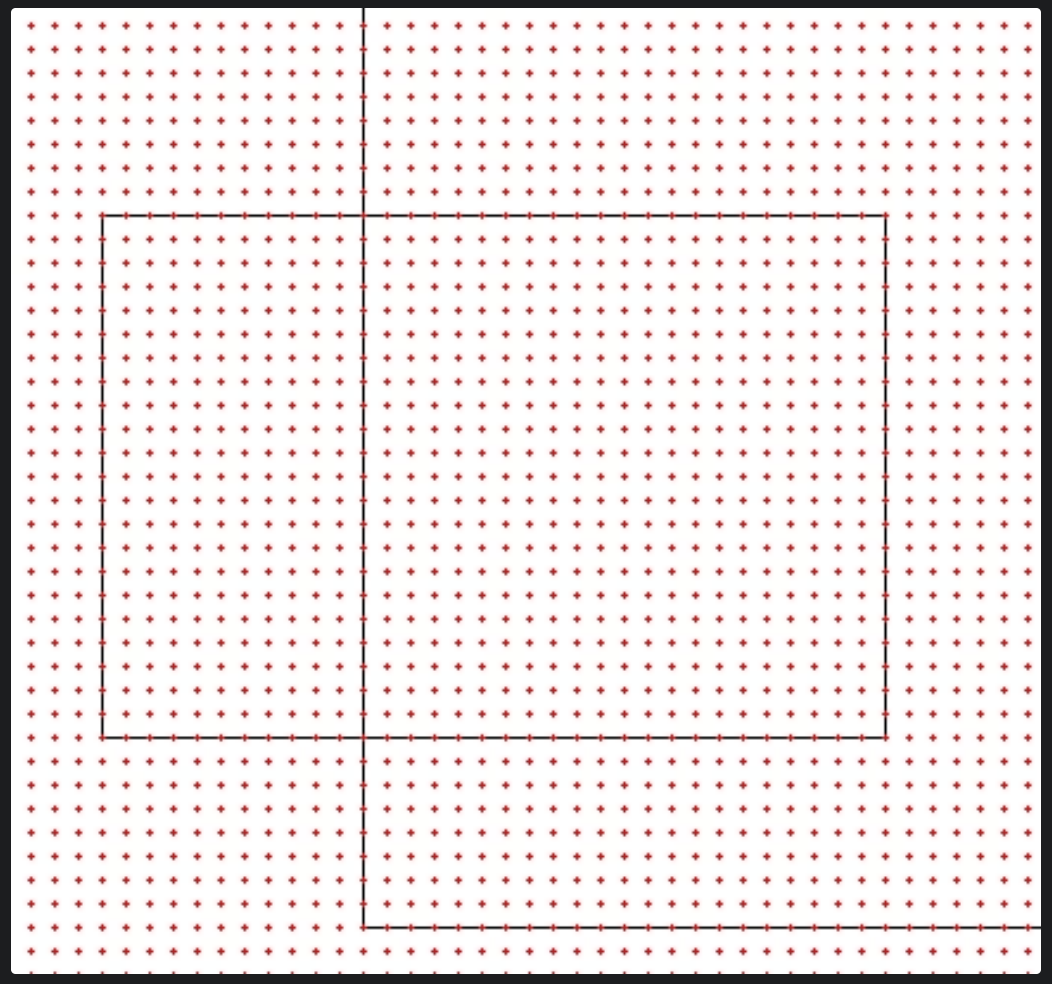
Made with GAMMA

Площадь объединения фигур

Черепахе был дан для исполнения следующий алгоритм:
Повтори 8 [Вперёд 22 Направо 90 Вперёд 33 Направо 90]
Поднять хвост
Назад 8 Направо 90 Вперёд 11 Налево 90
Опустить хвост
Повтори 8 [Вперёд 73 Направо 90 Вперёд 62 Направо 90]
Определите площадь области объединения фигур, построенных в результате выполнения заданного алгоритма.

Решение:

```
from turtle import *
tracer(0)
c = 10
left(90)
for i in range(8):
    forward(22*c)
    right(90)
    forward(33*c)
    right(90)
up()
back(8*c)
right(90)
forward(11*c)
left(90)
down()
for i in range(6):
    forward(73*c)
    right(90)
    forward(62*c)
    right(90)
up()
for x in range(-50,50):
    for y in range(-50,50):
        goto(x*c,y*c)
        dot(3,'red')
done()
```



Вычисление:

Площадь каждой фигуры:

- Первая фигура: 22 × 33 = 726
- Вторая фигура: 73 × 62 = 4526

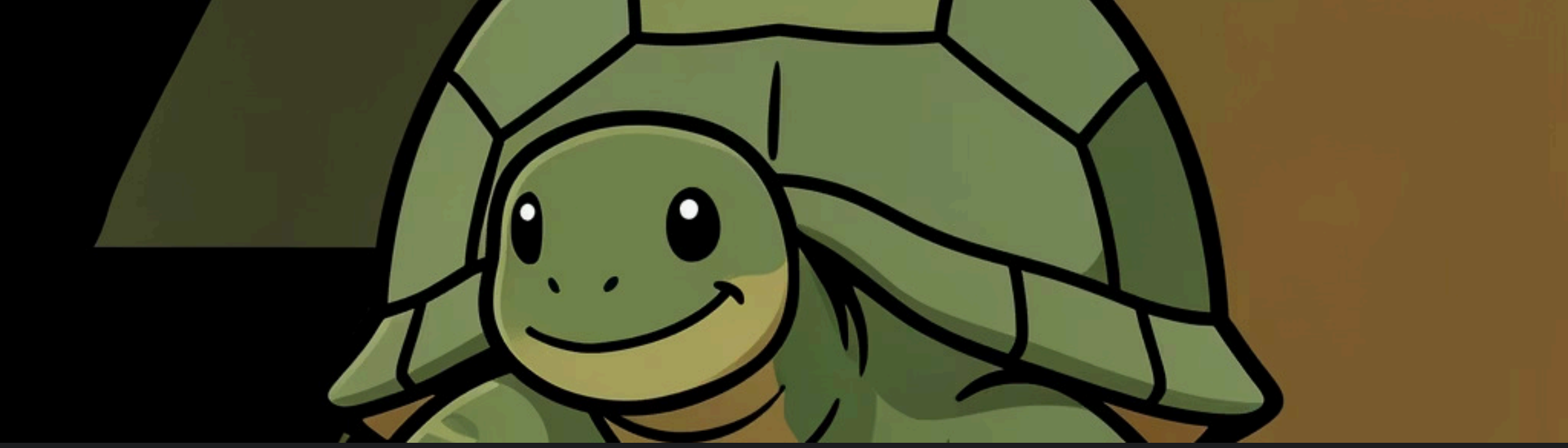
Площадь пересечения: 22 × 22 = 484

Площадь объединения:
726 + 4526 − 484 = 4768

Ответ: 4768

Примечание: На ЕГЭ встречаются задачи на подсчёт площади пересечения фигур. Они решаются аналогично задачам на периметр пересечения: сначала определяются размеры пересечения двух фигур, а затем вычисляется площадь этого пересечения путём умножения его длины на ширину. Например, если пересечение имеет размеры 18 × 30, то его площадь будет 18 × 30 = 540 квадратных единиц.

Made with GAMMA



Практические советы и частые ошибки

Направление черепахи

Всегда отслеживайте текущий угол поворота. Неправильный угол — самая частая ошибка, приводящая к незамкнутым фигурам

Масштабирование

Используйте коэффициент масштаба для удобства визуализации. Например, умножайте все координаты на 10 для лучшей видимости

Penup/Pendown

Не забывайте поднимать "хвост" penup() для перемещения без рисования и опускать pendown() перед рисованием новой части

Тестирование

Проверяйте код на простых примерах (квадрат, треугольник) перед переходом к сложным фигурам

> *"Лучше потратить 5 минут на проверку простого случая, чем час на поиск ошибки в сложном алгоритме"*

Made with GAMMA

Как эффективно подготовиться к заданию 6 ЕГЭ

Изучите основы

Освойте базовые команды turtle: forward, backward, left, right, penup, pendown, goto

Разберите 7 типов

Проработайте каждый тип задач с конкретными примерами кода и различными вариациями

Практикуйтесь

Решайте задачи из реальных вариантов ЕГЭ, визуализируйте результаты, проверяйте ответы

Обретите уверенность

Регулярная практика и систематический подход — ваш ключ к успешной сдаче экзамена!

7

Типов задач

Полный охват всех возможных вариантов

100

Практика

Решите минимум 100 задач для уверенности

1

Балл

Каждое правильное решение приближает к цели

Made with GAMMA